

Consuming the Sentinel Camera Grid

Integrator's guide for AI inference and dashboards. Each endpoint behaves like a physical camera on an operational network.

Access model: **HLS** is served over the CDN host (`cctv.corp8.cloud`), behind your access password) and works on any network. **RTSP & WebRTC/WHEP** carry media over TCP/UDP that a CDN cannot proxy, so they are served **directly on the public static IP** `103.250.160.189` (or a dedicated non-proxied subdomain such as `stream.corp8.cloud`). They are reachable from anywhere the gateway ports (8554/TCP, 8889/TCP, 8189/UDP) are open. **Credentials required:** RTSP & WebRTC now authenticate every connection with your **registered email and access password** embedded in the URL — `rtsp://email:password@...`. The `@` in your email must be percent-encoded as `%40` (e.g. `alice%40example.com`). Only emails on the approved access list can connect.

1 What you are connecting to

Every camera is published as a live stream. One second of video takes one second to arrive, frames carry monotonic presentation timestamps (PTS), and there is **no seeking and no way to run ahead of real time**. Treat each endpoint as a physical camera.

Protocol	Endpoint	Reachable via	Intended for
HLS	<code>https://cctv.corp8.cloud/<id>/index.m3u8</code>	Public (password)	Dashboards, mobile, restricted networks, remote AI
RTSP	<code>rtsp://<email>:<password>%40103.250.160.189:8554/stream/<id></code>	Public IP (direct)	AI inference (OpenCV, GStreamer, FFmpeg, DeepStream)
WebRTC (WHEP)	<code>http://<email>:<password>%40103.250.160.189:8889/stream/<id>/whep</code>	Public IP (direct)	Low-latency browser preview

`<id>` is the camera id (`cam01` ... `cam30`). Start from the catalogue rather than hard-coding — the camera set can change:

```
curl -s https://cctv.corp8.cloud/cameras.json
```

2 Connecting

OpenCV (Python) — HLS (remote) or RTSP (on-network)

```
import os, cv2
# HLS (works anywhere with your session):
cap = cv2.VideoCapture("https://cctv.corp8.cloud/cam04/index.m3u8", cv2.CAP_FFMPEG)
# RTSP (on LAN/Tailscale – force TCP):
os.environ["OPENCV_FFMPEG_CAPTURE_OPTIONS"] = "rtsp_transport;tcp"
cap = cv2.VideoCapture("rtsp://you%40example.com:YOUR-PASSWORD@103.250.160.189:8554/stream/cam04")
while True:
    ok, frame = cap.read()
    if not ok: break          # reconnect – see §3
    pts_ms = cap.get(cv2.CAP_PROP_POS_MSEC)
```

GStreamer (RTSP)

```
gst-launch-1.0 rtspsrc location=rtsp://you%40example.com:YOUR-PASSWORD@103.250.160.189:8554/stream/cam04 ! rtph264depay ! h264parse ! avdec_h264 ! videoconvert ! fakesink
```

FFmpeg / ffprobe

```
ffplay -rtsp_transport tcp rtsp://you%40example.com:YOUR-PASSWORD@103.250.160.189:8554/stream/cam04
ffplay https://cctv.corp8.cloud/cam04/index.m3u8
```

3 Do's and don'ts

DO — Force RTSP over TCP. UDP fails across NAT/firewalls and produces corrupt frames that look like model bugs. Set `rtsp_transport=tcp`. If 8554 is blocked, use the HLS endpoint.

DON'T — Trust the reported frame rate. `CAP_PROP_FPS` often mismatches real delivery. Use timestamps for any speed/dwell/time metric.

DO — Drive timing from PTS, never arrival time. On connect, a buffered GOP replays so the first second can arrive faster than real time. Feed trackers PTS deltas.

DON'T — Assume a constant frame rate. Inter-frame gaps are normal; tolerate them without treating them as a disconnect.

DO — Reconnect with backoff. Feeds are supervised and may restart. Exponential backoff (~2s → cap ~30s). Never tight-loop.

DON'T — Treat join-time decode warnings as fatal. Mixed H.264/H.265 may log `Could not find ref with POC` until the first IDR. Normal, self-corrects.

DO — Expect a scene discontinuity. Each feed loops; at the loop point the scene cuts abruptly. Long-lived state must recover from a hard cut.

DON'T — Plan on downloading footage. There is no file download; the grid is consumed live. Build against a live capture from the start.

DO — Pace your load. Each client gets its own stream copy. Open only the cameras you are processing; close captures you finish with.

4 Pre-submission checklist

- ☐ RTSP clients force TCP; remote clients use HLS.
- ☐ No timing logic depends on `CAP_PROP_FPS` or frame arrival time.
- ☐ Inter-frame gaps do not crash or stall the pipeline.
- ☐ Reconnect with backoff is implemented and tested.
- ☐ Decoder warnings on join are logged, not fatal.
- ☐ Camera list read from `cameras.json`; mixed H.264/H.265 and resolutions handled.
- ☐ Behaviour is sane across a scene discontinuity (loop point).

5 Support

Report feed problems with the camera id, exact URL, your client + version, UTC timestamp, and the client-side error log. Confirm the camera's status on the live grid before reporting it down.

Sentinel Camera Grid · Integrator resources · authorised access only